

Codebase Technical Audit

Project: GEngine **Repository:** C:\dev\GEngine **Audited revision:** 1fa379c0f988f72e460b14478fcb637f0ea370bd (feature/imgui_editor) **Audit date:** 2026-08-30 **Authoritative artifact:** this Markdown file

Scope note: build and runtime tests were performed against a clean export of the audited commit. Source analysis used the current working tree so that the owner's in-progress rendering changes were not ignored. Those pre-existing changes are listed under "Audit basis and limitations." No production source was modified by this audit.

Executive Summary

GEngine is a Windows-first C++ graphics/physics sandbox built as one static engine library plus four console applications: Breakout, GEngineEditor, RayTracing, and RigidBodySimulation. It uses SDL2/OpenGL 4.6, GLAD, ImGui, GLM, EnTT, Assimp, FMOD, TBB, and a mixture of older scene-graph rendering and newer ECS rendering. The repository is generated with Premake, not CMake.

The clean audited revision **generates and compiles successfully in both Debug and Release on x64 Windows** with Visual Studio's v143 MSVC toolchain. Release completed with 0 errors and 956 warnings. MSVC code analysis on the engine target completed with 0 errors and 266 warnings. There is no automated test target or test framework.

The build is not yet runnable from a clean checkout: all four Debug and Release executables reach SDL/ImGui initialization, then assert because `ImGuiWindow` loads fonts through a relative path that is neither valid from the documented executable working directory nor staged by a working post-build action. This is a repository packaging/path defect, not a missing system dependency.

The most urgent correctness issues are:

- `_Scene` indexes fixed five-element vectors using an opaque OpenGL program object minus one; program names are not dense scene indices, so ordinary shader creation can cause out-of-bounds memory corruption.
- Breakout and GEngineEditor free global asset/shader managers in derived destructors and `BaseApp` frees them again; the manager maps are not cleared, producing double deletion on normal shutdown once startup succeeds.
- Several independently reachable paths produce undefined behavior or invalid GPU work: uninitialized mouse relative-mode state, missing non-void returns, incorrect vertex upload byte counts, invalid multisample blit filtering, and zero-vector normalization in collision code.
- The main loop's minimized branch never pumps events, and its time conversion/pacing logic passes an invalid unit to input while discarding real elapsed time. Physics therefore is frame-rate dependent and restoration from minimize can busy-spin indefinitely.
- The current rendering configuration allocates two $8192 \times 8192 \times 6$ depth resources—approximately 3 GiB combined before normal render targets—and redraws the entire scene for point shadows, cascaded shadows, picking, and the final pass every frame.

6. Resource ownership is inconsistent. `Texture` does not delete its OpenGL object, physics shapes are unowned leaks, an `Actor` move assignment deletes a non-owning parent, and SDL destroys the graphics context before shutting down ImGui's OpenGL backend.

The recommended first milestone is not a rendering optimization. It is a small correctness release that repairs the ECS grouping invariant, gives managers idempotent/RAII ownership, fixes startup asset resolution, corrects input and buffer undefined behavior, and establishes a deterministic shutdown order. Only after that foundation should shadow sizing, render-pass scheduling, physics stepping, and allocation behavior be optimized.

Audit result at a glance

Area	Result
Clean project generation	Pass: bundled Premake generated Visual Studio 2022 files
Clean Debug build	Pass, x64, MSVC v143
Clean Release build	Pass, x64, MSVC v143; 956 warnings
Application smoke test	Fail for all four apps in both configurations: font assertion
Existing automated tests	None found
MSVC code analysis	Pass with 266 warnings; useful findings include C6255, C4715, C26800
CMake	No project-level CMake build; only third-party GLM/SDL CMake files
clang-tidy	No configuration and executable unavailable
Threading	TBB CPU parallelism in ray tracer/containers; OpenGL work remains main-threaded
Highest severity	2 Critical confirmed bugs, followed by multiple High confirmed bugs

Repository Overview

Inventory

The audited checkout contains 1,513 tracked files and is approximately 865 MiB. The unusually large size is primarily engine assets (about 473 MiB), committed runtime binaries/build outputs (about 284 MiB), and demonstration GIFs (about 86 MiB).

Path	Role	Notable contents
<code>GEngine/include/GEngine</code>	Public headers plus implementation files	Core, renderer, ECS, physics, math, mesh, material, animation, audio, windows, assets
<code>GEngine/src</code>	Engine implementation	Static library implementation and platform-specific SDL/OpenGL window code
<code>Breakout</code>	Executable	Legacy Actor/Scene 2D game sample
<code>GEngineEditor</code>	Executable	Legacy scene/editor and ImGui hierarchy panel
<code>RayTracing</code>	Executable	CPU ray tracer displayed through the engine window/UI
<code>RigidBodySimulation</code>	Executable	EnTT ECS, PBR renderer, shadows, picking, and custom rigid-body physics
<code>external</code>	Third-party source/prebuilt dependencies	Assimp, EnTT, FMOD, GLAD, ImGui, reflection, SDL2, spdlog, TBB; <code>rttr</code> is empty
<code>GEngine/include/external</code>	Header/source dependencies embedded in engine include tree	GLM, ImGui, eventpp, stb_image
<code>vendor/bin/premake</code>	Build generator	Premake binaries for Windows, Linux, and macOS

Path	Role	Notable contents
bin	Committed runtime/build artifacts	Executables, DLLs, libraries, PDBs by configuration/application
bin-int, x64, .vs	Generated artifacts	Intermediate/object/IDE state; some are present in the working checkout
tools	Build/run helpers	solution generation, MSBuild wrapper, run wrapper, symlink helper
gif	Documentation assets	renderer and physics demonstrations

The dominant extensions are 558 `.h`, 299 `.hpp`, 147 `.inl`, and 136 `.cpp` files. The first three counts are inflated by vendored header-only libraries. Engine code also places compilable `.cpp` files below `GEngine/include/GEngine`; Premake explicitly compiles that subtree. That organization makes public-interface boundaries and ODR safety harder to reason about.

Products and entry points

Product	Entry file	App class	Primary subsystem
Breakout	<code>Breakout/src/main.cpp</code>	<code>BreakoutApp</code>	legacy 2D renderer/game logic/audio
GEngineEditor	<code>GEngineEditor/src/main.cpp</code>	<code>SceneApp</code>	legacy scene renderer/editor
RayTracing	<code>RayTracing/src/main.cpp</code>	<code>RayTracing</code>	TBB-parallel CPU path/ray tracing
RigidBodySimulation	<code>RigidBodySimulation/src/main.cpp</code>	<code>RigidBodySimulation</code>	ECS renderer + custom 3D physics

Each `main.cpp` includes `GEngine/Core/EntryPoint.h`, which supplies the actual `main` at lines 16–31 and invokes the product-specific `CreateApp()` factory.

Third-party dependency map

Dependency	Use	Form in repository
SDL2 / SDL2_ttf	window, OpenGL context, input, font support	headers, libraries, committed DLLs
GLAD	OpenGL loader	source/static library
GLM	vectors, matrices, quaternions	vendored headers/source tree
ImGui	editor/debug UI and platform viewports	vendored source under engine include tree
ImGuizmo	editor transform gizmos	external source
EnTT	ECS registry/view/group implementation	vendored header
Assimp	model/animation import	headers, libraries, DLLs
FMOD / FMOD Studio	audio	headers, Debug/Release libraries and DLLs
TBB	CPU parallel loops/containers	headers, libraries, runtime
spdlog/fmt	logging	vendored headers
eventpp	event dispatch support	vendored headers
stb_image	image decode	vendored implementation
RTTR/reflection	reflection experiments	<code>external/reflection</code> ; <code>external/rttr</code> is empty

Runtime directories contain SDL2, SDL2_ttf, FMOD/FMOD Studio, Assimp, FreeType, zlib, and TBB-related binaries as applicable. Premake currently links both debug and release library variants in each executable's shared link list rather than selecting one by configuration.

Platform-specific code

Windows-specific SDL/OpenGL code lives in `GEngine/src/Windows` and `GEngine/include/GEngine/Windows`. Premake contains Linux and macOS filters, but the effective project is Windows-only today:

- `EntryPoint.h:19` calls `_CrtSetDbgFlag` without a Windows/compiler guard.
- Dependency paths and committed libraries are Windows `.lib/.dll` artifacts.
- `tools/buildsln.py:29` hard-codes a Visual Studio Community MSBuild path.
- OpenGL context creation requests version 4.6 (`SDLWindow.cpp`), and shaders include GLSL 4.60 sources.
- No non-Windows dependency acquisition or validated compiler/link configuration exists.

The Linux/macOS Premake filters should be considered aspirational rather than supported targets.

Audit basis and limitations

The worktree already contained owner changes in five shaders, `RenderSystem.cpp/.h`, `RenderTarget.h/.cpp`, `BaseApp.cpp`, and `RigidBodySimulation` files. They were preserved. Build and launch evidence uses a clean export of commit `1fa379c0...`; source observations explicitly called “current working tree” include those in-progress framebuffer/renderer changes. No GPU frame capture or benchmark could be collected because clean startup stops at font initialization. Performance statements are therefore either code-proven counts/allocations or explicitly classified opportunities/risks, not measured speedups.

Build System

Authoritative build definition

There is **no root CMakeLists.txt**, **no CMake preset**, and **no engine/application CMake target**. The only CMake inputs belong to bundled dependencies (`GEngine/include/external/glm/CMakeLists.txt` and SDL CMake templates). Running CMake at the repository root is not a supported build path.

`premake5.lua` is authoritative:

- workspace `GEngine`, x64, Debug/Release (`premake5.lua:1-9`);
- static libraries `GEngine` and `GLAD`;
- C++20 requested in Premake (`premake5.lua:33-38`), although generated Visual Studio projects use `/std:c++latest`, not a pinned `/std:c++20`;
- precompiled header `gepch.h/gepch.cpp` (`premake5.lua:43-44`);
- four `ConsoleApp` projects linked to the engine;
- output layout `bin/<configuration>/<project>` and `bin-int/<configuration>/<project>` (`premake5.lua:11-12`);
- Windows filters define `GE_PLATFORM_WINDOWS`, `SDL_MAIN_HANDLED`, and OpenGL/SDL dependencies.

Compiler and configuration settings

Setting	Debug	Release	Evidence/notes
Architecture	x64	x64	workspace architecture
Language mode	generated <code>/std:c++latest</code>	generated <code>/std:c++latest</code>	broader than requested C++20
Runtime	<code>/MTd</code>	<code>/MT</code> in generated project	Premake contains unconditional <code>/MTd</code> build options at lines 93, 215, 373, 530, 692; generated runtime property wins, but definitions are contradictory
Symbols	on	off/optimized	Premake configuration filters

Setting	Debug	Release	Evidence/notes
Optimization	off	speed	Premake configuration filters
Warnings as errors	no	no	hundreds of warnings allowed
Platform toolset	v143	v143	clean MSBuild results

systemversion "lastest" at premake5.lua:96 is misspelled. It did not block the tested generator/toolchain but is not a reliable request for the latest Windows SDK.

Dependency configuration concerns

The project links configuration-specific libraries in an unconditional list. For example, executable sections include TBB/FMOD debug and release library names together (premake5.lua:234–249, 392–407, 550–564, and the analogous RigidBodySimulation block). Release output also stages both debug and release Assimp/FMOD DLL variants. This increases ambiguity and can load/link an unintended runtime.

Post-build commands refer to PostBuildCopy or PostBuildCopy_windows locations that do not exist in the clean checkout. Their robocopy failures do not fail MSBuild, so "build succeeded" does not imply a runnable package. The checked-in bin directory masks this problem for non-clean development trees.

Build results

Operation	Result	Diagnostics
Premake vs2022 generation	Success	solution/projects generated from clean export
Full Debug x64 build	Success	0 errors
Full Release x64 build	Success	0 errors, 956 warnings, about 1m34s
MSVC engine rebuild with code analysis	Success	0 errors, 266 warnings, about 2m46s

The first isolated build attempt in an OS temporary directory failed because MSVC FileTracker was denied by the managed sandbox and because the host exposed inconsistent Path/PATH variables. Re-running the exact clean build inside the writable workspace with a normalized process PATH succeeded. Those two failures are environment/configuration issues, not repository failures.

Repository vs dependency vs environment problems

Type	Items
Repository problems	no root CMake despite CMake-like dependency files; broken asset staging; ignored post-build copy failures; contradictory runtime flags; unconditional Debug+Release links; hard-coded helper paths; Windows-only code behind multi-platform filters; warning debt; duplicate compiled definition
Missing dependencies	none for the tested Windows compile/link; optional clang-tidy, pandoc, and Python PDF packages were absent; empty external/rttr appears unused by active targets
Environment/configuration	Visual Studio/MSVC and a desktop/OpenGL 4.6-capable environment are required; sandbox FileTracker and duplicate PATH needed local normalization; runtime graphics testing requires interactive window support

Build Instructions

Clean checkout: supported Windows path

Prerequisites: Windows x64, Visual Studio 2022 with Desktop development with C++ and Windows SDK, Python 3 for helper scripts, and Git. No dependency download was required in the audited revision.

From a Developer PowerShell at the repository root:

```
# 1. Generate Visual Studio 2022 projects using the committed generator.
& .\vendor\bin\premake\premake5.exe vs2022

# 2a. Build Debug.
msbuild .\GEngine.sln /m /nologo /p:Configuration=Debug /p:Platform=x64

# 2b. Build Release.
msbuild .\GEngine.sln /m /nologo /p:Configuration=Release /p:Platform=x64
```

If `msbuild` is not on `PATH`, call the VS installation's `MSBuild\Current\Bin\amd64\MSBuild.exe`. The repository's `python cli.py build` route is less reproducible: `tools/buildsln.py` hard-codes one VS2022 Community path and uses lower-case defaults, while `cli.py:14` defaults to Release. Prefer the explicit commands above until those helpers are repaired.

CMake status

There is no valid command such as `cmake -S . -B build`; adding a root CMake build would be a new build-system project, not a missing invocation. If CMake support is desired, define targets for GLAD, GEngine, all four applications, imported configuration-specific binaries, asset staging, and platform checks, then validate it independently rather than treating third-party CMake files as project support.

Run Instructions

Intended invocation

`tools/run.py` selects an application (default `GEngineEditor`), and `tools/run.bat` changes into `bin/<configuration>/<application>` before launching it. There are no application command-line arguments; `argc` and `argv` are not consumed.

Equivalent manual commands are:

```
Push-Location .\bin\Debug\RigidBodySimulation
.\RigidBodySimulation.exe
Pop-Location
```

Replace application/configuration with one of:

- Breakout
- GEngineEditor
- RayTracing
- RigidBodySimulation

The executable directory is the assumed working directory. Runtime DLLs must sit beside the executable, and the code also assumes relative access to shaders, textures, models, audio, and fonts.

Actual clean-checkout result

All eight smoke tests (four applications × Debug/Release) failed during ImGui font initialization.

GEngine/src/Windows/ImGuiWindow.cpp:10–11 requests:

```
../GEngine/include/GEngine/Assets/Fonts/OpenSans-Regular.ttf
../GEngine/include/GEngine/Assets/Fonts/OpenSans-Bold.ttf
```

From `bin/Debug/<Application>`, that resolves below `bin/Debug/GEngine/...`, which does not exist in a clean package. ImGui asserts at `imgui_draw.cpp:2161` with “Could not load font file!”. The proper fix is to establish a runtime asset root (preferably executable-relative or configured), stage assets in a build target that fails on copy errors, and have every loader use that root. Merely changing the working directory would break other existing relative assumptions and is not a durable fix.

Runtime initialization dependencies

- SDL video and game-controller subsystems;
- an OpenGL 4.6 core-capable driver/context;
- SDL2_ttf and font assets;
- the DLL set staged beside each executable (SDL2, SDL2_ttf, Assimp, FMOD, FreeType/zlib, and TBB where required);
- application/engine shader, texture, model, font, and audio assets at expected relative paths.

Application Execution Flow

Initialization sequence

```
OS process
-> EntryPoint.h::main
  -> _CrtSetDbgFlag (currently unconditional MSVC/Windows call)
  -> application CreateApp()
  -> BaseApp-derived constructor
  -> BaseApp::Initialize
    -> GEngine::Initialize
      -> Log
      -> SDL_Init(video | game controller)
      -> WindowManager::Initialize / SDLWindow / GL context / GLAD
      -> InputManager::Initialize
      -> EventManager::Initialize
      -> ShapeManager::Initialize
      -> TTF_Init
    -> legacy Scene and event handlers
    -> RenderTarget + shadow targets + picking/final FBO + UBO
  -> BaseApp::Run
    -> OnCreated (application content)
    -> per-frame input/update/render/UI/present
  -> delete application
  -> derived destructor, then BaseApp destructor
  -> process/static manager destruction
```

`GEngine::Initialize` is at `GEngine/src/Core/GEngine.cpp:43–105`. `BaseApp::Initialize` is at `GEngine/src/Core/BaseApp.cpp:58–134`, and the loop begins at line 274. Initialization assumes the primary SDL window has ID 1 (`BaseApp.cpp:118`), which is an implicit global invariant rather than a returned handle.

Main loop

The intended steady-state flow is:

```

clock_delta -> InputManager::Update
             -> EventManager::Update (SDL_PollEvent)
             -> application ProcessInput
             -> application Update / physics
             -> legacy Scene::Update
             -> window BeginRender / app Render / ImGui
             -> window EndRender -> SDL_GL_SwapWindow

```

The actual timing code computes microseconds, multiplies by 1,000 while naming the result milliseconds (`BaseApp.cpp:293–315`), sleeps toward 16 ms, then forces the simulation delta to 16,000 μ s regardless of oversleep. Deltas over 33 ms are discarded rather than accumulated. The `RigidBodySimulation` divides the chosen delta into two substeps (`RigidBodySimulation.cpp:663–668`), but this is still variable/clipped stepping, not a fixed accumulator.

If `m_Minimized` is true, the loop immediately continues at `BaseApp.cpp:285` before event polling or sleeping. The application can therefore consume a full CPU core and cannot observe its restore/quit event.

Shutdown order

The intended order should be application/ECS/physics resources → GPU resources while context current → ImGui OpenGL/SDL backends → GL context → SDL window/controller → TTF/SDL shutdown. The current order is inconsistent:

- derived applications may free singleton manager resources;
- `BaseApp::~~BaseApp` frees them again (`BaseApp.cpp:26–31`);
- `SDLWindow::Shutdown` destroys the SDL window at line 109 and GL context at line 112, then deletes `ImGuiWindow` at line 114; its destructor shuts down ImGui backends after their context is gone;
- no `SDL_Quit`, `TTF_Quit`, or game-controller close path was found.

Architecture

Major modules and responsibilities

Module	Responsibility	Owns/depends on
Core/GEngine	subsystem bootstrap	singleton-style managers, SDL/TTF
Core/BaseApp	application lifecycle, loop, common targets	GEngine, legacy Scene, cameras, FBOs, UBO
Windows	SDL window/context and ImGui integration	SDL2, GLAD, ImGui backends
Managers	windows, input, events, assets, shaders, shapes	global/static containers and raw resource pointers
legacy Core/Scene, Actor, Group	hierarchy/update/render organization	raw parent/child relations, renderers/materials
ECS Scene/_Scene, _Entity, components	entity registry, render/light grouping, physics component startup	EnTT, PhysicsWorld, render-system grouping
Core/Renderer / Renderer2D	legacy draw submission	Shader, Geometry, Material, GL state
Core/RenderSystem	ECS PBR/shadow/picking submission	EnTT registry, GL, components, render targets
Core/RenderTarget, FrameBuffer	FBOs, multisampling, shadow maps, picking, UBO	OpenGL object IDs
Mesh, Material, Assets	VAO/VBO/EBO, uniforms, textures, models	OpenGL + Assimp/stb_image
Physics	bodies, broadphase, GJK/contact/manifold, constraints	GLM and custom dynamic matrix/vector math
Animation	bone interpolation/skinning	Assimp/GLM

Module	Responsibility	Owns/depends on
Audio	FMOD systems/events	FMOD runtime
SimpleRenderer, ray-tracing camera/scene	CPU ray tracing	TBB, image buffer, engine presentation

Two rendering/scene architectures

The repository is midway through an architectural transition:

Legacy path ----- Actor -> Group -> Scene Geometry + Material -> Renderer OpenGL Breakout, GEngineEditor	ECS path ----- _Entity -> entt::registry -> _Scene Components -> RenderSystem groups OpenGL multipass PBR/shadows/picking RigidBodySimulation
---	--

RayTracing is a third, mostly independent path: `SimpleRenderer` computes pixels on the CPU in parallel, uploads/displays an image through the engine, and uses the common SDL/ImGui application shell.

Maintaining both legacy and ECS renderers duplicates light handling, GL state setup, scene ownership, and bugs—the incorrect `SpotLight` component access exists in both. A long-term architecture should choose one scene/render submission model or put both behind a shared immutable render-world extraction stage.

Ownership and lifetime model

Ownership is largely conventional rather than encoded:

- manager maps store owning raw `Texture`, `Font`, and `Shader*`;
- scene actors and ECS physics components store raw pointers with mixed ownership;
- `_Scene` owns a raw `PhysicsSystem` and creates a raw `PhysicsWorld`;
- `PhysicsWorld` deletes rigid bodies but bodies do not delete their shapes;
- parent pointers are documented non-owning but a move assignment deletes one;
- OpenGL wrappers inconsistently provide destructors.

This makes shutdown order and replacement operations unsafe. The target model should use `unique_ptr` for exclusive CPU ownership, stable non-owning handles/IDs for relationships, explicit shared asset handles where caching is intended, and move-only RAII wrappers for every GL object type.

Rendering Architecture

Rendering API abstraction

OpenGL is not fully abstracted. Classes such as `Shader`, `Texture`, `Geometry`, `VertexBuffer`, `IndexBuffer`, `FrameBuffer`, `RenderTarget`, and `UniformBufferObject` wrap some object operations, but `Renderer`, `RenderSystem`, application code, and component bind methods all issue direct `gl*` state calls. There is no command buffer, render graph, central state cache, or backend-neutral interface.

Consequences include:

- ownership policy differs by wrapper;
- framebuffer validation and resizing are duplicated;

- per-entity code repeatedly changes global state;
- pass dependencies are implicit in call order;
- it is difficult to count, batch, or test submissions without a live GL context.

Assets, shaders, meshes, and materials

`ShaderManager::LoadShader` caches `Shader*`; `Shader` compiles/links on first load and caches uniform locations. There is no evidence of per-frame shader recompilation. There is a redundant second `FindUniformLocations` call after link (`ShaderManager.cpp:24`, following `Shader::Link`), but it is a startup/load cost.

The asset texture cache is broken: `AssetsManager::LoadTexture` looks up the full `image_dir` at `AssetsManager.cpp:25` but inserts using `imageFilePath` at line 35. Repeated loads miss, allocate another texture, and may fail map insertion while returning an untracked raw pointer. In addition, `Texture` declares no destructor that calls `glDeleteTextures`, so even tracked wrapper deletion leaks GPU texture storage.

Geometry owns VAO/buffer wrappers. `VertexBuffer::SetData` passes `data.size()` to `glBufferSubData` (`VertexBuffer.cpp:45`) rather than `data.size() * sizeof(float)`, uploading only one quarter of typical float data. Geometry also deletes buffer IDs directly while `IndexBuffer` deletes its ID, causing a redundant double `glDeleteBuffers` call for the index buffer.

Materials are responsible for per-object uniforms and texture binding. `ECS TexturesComponent::BindTextures` iterates an unordered container and binds all textures for each entity/pass. No material/texture sort minimizes changes.

Framebuffers and render targets

The current working tree constructs:

- a multisampled scene render target;
- layered cascade depth target;
- depth cubemap for point shadows;
- integer mouse-picking framebuffer;
- final multisampled color/depth-stencil framebuffer;
- a uniform buffer for shared camera/light matrices.

Current `RenderTarget.cpp` contains several concrete faults:

- `BindAndBlitToScreen` uses `GL_LINEAR` in a multisample resolve (`RenderTarget.cpp:408–414`); multisample resolve requires `GL_NEAREST`.
- the generic target configures draw attachments 0 and 1 while attachment 1 creation is commented out (`RenderTarget.cpp:236–250`), making the framebuffer incomplete.
- mouse-pick and final textures allocate `width × width`, not `width × height` (`RenderTarget.cpp:769, 873, 884`).
- the integer picking texture is configured with linear filtering; integer textures require nearest filtering.
- `InvalidateMousePickProcessing` binds a framebuffer using a texture ID before framebuffer creation (`RenderTarget.cpp:361–385`).
- the multisample anisotropy branch binds `GL_TEXTURE_2D` rather than the selected multisample target (`RenderTarget.cpp:220`).
- size validation is commented out, so minimized/invalid viewport dimensions can drive allocations.

The current `RigidBodySimulation` initializes both cascade and point shadow maps at 8192×8192 with six layers/faces (`BaseApp.cpp:123–124`). At 32-bit depth this is roughly 1.5 GiB per resource, or 3.0 GiB combined, excluding color, mesh, texture, driver, and mip/storage overhead. This can exceed practical VRAM budgets and cause allocation failure or severe paging.

Frame Rendering Pipeline

Complete RigidBodySimulation frame trace

The ECS renderer is the most feature-complete frame and is traced here from `BaseApp::Run` into `RigidBodySimulation.cpp:726-850`:

```
1. BaseApp clock/input/events/update
2. RigidBodySimulation physics: two Update(dt/2) substeps
3. Reset renderer statistics
4. SetupUBO
   camera projection/view/position + light-space matrices
   six glBufferSubData updates
5. Set default depth/cull/polygon state
6. Point shadow pass
   bind 8192² depth cubemap FBO
   for each render group/entity:
     bind depth shader -> VAO -> model transform -> draw
   geometry shader emits six cubemap faces
7. Cascaded directional shadow pass
   bind 8192² layered depth FBO
   repeat scene geometry
   geometry shader emits five cascades
8. Mouse-picking pass
   bind integer ID FBO
   repeat entire scene every frame
9. Final scene pass
   bind multisampled final FBO
   for each shader group/entity:
     bind program
     update render settings
     upload transform/material/light uniforms
     bind textures and VAO
     glDrawElements/glDrawArrays
10. Point-light visualization
11. Skybox
12. Resolve multisample FBO
13. Build/render ImGui UI and optional platform windows
14. SDL_GL_SwapWindow -> driver/compositor presents
```

The pass call order is hard-coded in the application rather than represented as data. A visible mesh is normally submitted four times per frame before light-helper/UI draws. Geometry shaders multiply shadow work across six point-light faces and five cascade layers. No visibility culling or pass-specific static cache was found in the submission path.

State changes and draw submission

`RenderSystem::UpdateRenderSetting` (`RenderSystem.cpp:843-865`) unconditionally sets culling, polygon mode, and line width for each entity. Shader grouping removes some program binds, but there is no cache for VAO, blend, cull, depth, polygon, line width, or texture state and no sort by material/texture. Point light visualization (`RenderSystem.cpp:1056-1081`) iterates all point lights inside every non-empty light group, duplicating visualization draws as group count grows.

`SetupUBO` builds matrix vectors and performs six separate `glBufferSubData` uploads each frame (`RenderSystem.cpp:823-831`), while its backing buffer is created with `GL_STATIC_DRAW`. A packed per-frame structure and a dynamic/persistent upload strategy would reduce driver calls and communicate the actual usage.

Picking and synchronization

The picking pass renders all pickable geometry every frame even though selection readback occurs only after a click. `glReadPixels` in the click path is synchronous and can stall until earlier rendering completes. Render the ID buffer on demand (or only when mouse/camera/scene state invalidates it); if frequent hover picking is needed, use a small scissored region or PBO/fence pipeline. Ensure viewport-to-FBO coordinate scaling and Y inversion use actual width and height after fixing the square-allocation defect.

Resize handling

Framebuffer resize methods destroy/recreate attachments synchronously. The current editor viewport code compares ImGui panel size with the native SDL window and calls both render-target/camera resize and `SDLWindow::OnResize` (`RigidBodySimulation.cpp:1106-1128`). This couples a docked panel to the OS window and can create resize feedback or repeated large reallocations. Native window size, framebuffer pixel size, and editor viewport size should be separate state, debounced, validated against zero, and used only by the resources that depend on each.

Shader correctness

`pbr_cascade_shadow.frag` samples a normal texture but leaves the required `[0,1] → [-1,1]` remap commented out (`pbr_cascade_shadow.frag:43-44`). It also ignores the function's tiled UV argument and samples `fs_in.TexCoords`, while other material channels use tiled coordinates. This biases normals toward positive tangent directions and misaligns normal detail from albedo/roughness/metalness.

The shader writes a constant entity ID 50 in the normal color pass. Picking uses a separate shader, so this output should be removed or explicitly wired to the intended integer attachment rather than suggesting valid per-entity data.

GPU Resource Lifetime

Current ownership map

Resource	Owner in code	Destruction status
Shader program	ShaderManager raw pointers	deleted by manager; map not cleared, so repeated free double-deletes wrappers
Image texture	AssetsManager raw pointers	wrapper deleted, GL texture not deleted
VAO/VBO/EBO	Geometry/buffer wrappers	partial RAII; index buffer is redundantly deleted
Generic/final/pick/shadow FBOs	BaseApp/RenderTarget wrappers	inconsistent across audited revision/current WIP; current changes add some destructors but policy remains fragmented
UBO	UniformBufferObject wrapper	no complete move-only RAII contract in audited revision
ImGui device objects	ImGuiWindow	shutdown occurs after SDL GL context deletion

Every GL wrapper should be non-copyable, move-enabled, store zero as “empty,” delete exactly once, and require destruction on the owning context thread before that context is torn down. Non-owning attachment views should use a distinct type or explicit ownership flag; otherwise adding a `Texture` destructor would make framebuffer attachment wrappers double-delete.

CPU/GPU synchronization and stalls

No explicit fences or persistent mapped buffers are used. The primary definite synchronous readback is mouse-pick `glReadPixels`. Synchronous FBO recreation during viewport resize and many `glBufferSubData` calls can serialize with in-flight use depending on the driver. Shadow maps are rewritten in full each frame. There is no evidence of accidental graphics calls from

TBB worker threads; the CPU ray tracer produces CPU image data and the main thread performs display-side graphics work.

Physics and Math Architecture

Update and data flow

```
BaseApp::Run selected/clipped dt
-> RigidBodySimulation::Update
    -> PhysicsWorld::Update(dt/2), twice
        -> apply gravity impulses
        -> broadphase SweepAndPrune1D
        -> narrow phase (sphere special case / GJK-style contact)
        -> contact resolution and/or manifolds
        -> constraint pre-solve / iterative solve / post-solve
        -> integrate body linear/angular state
    -> copy PhysicsBody position/orientation into ECS Transform3DComponent
-> render transform matrices
```

`_Scene::OnPhysics3DStart` creates shapes and rigid bodies from ECS components (`_Scene.cpp:438-527`). `PhysicsWorld` owns/deletes bodies (`PhysicsWorld.cpp:6-15`) but body shapes are raw pointers and are never deleted. `OnRuntimeStop` is empty (`_Scene.cpp:160-163`) even though `OnPhysics3DStop` exists at line 534, and repeated start replaces `m_PhysicsWorld` without deleting the previous world.

Timestep and integration

The world applies gravity as an impulse and integrates linear/angular state. For valid dynamic bodies, the gravity expression cancels mass and produces `g*dt`; however it first computes `mass = 1 / inverseMass` (`PhysicsSystem.cpp:169`), so zero inverse mass produces infinity before static-body checks inside later calls.

The outer loop is not a deterministic fixed timestep. It forces many frames to 16 ms, discards long frames beyond 33 ms, and does not retain remainder in an accumulator. The two `RigidBodySimulation` substeps improve solver behavior but inherit this time error. Use a monotonic elapsed-time accumulator, clamp only catastrophic backlog, run a fixed number/size of steps, retain remainder, and interpolate render transforms.

Dynamic collision tests temporarily integrate bodies to time of impact and then integrate with a negative time (`PhysicsSystem.cpp:331-343`). Semi-implicit linear/angular integration, normalization, and floating-point rounding are not exactly reversible. Collision queries should be pure: calculate predicted transforms without mutating live bodies, then advance the world once according to a documented continuous-collision policy.

Coordinate spaces and transforms

Physics body helpers consistently transpose `glm::toMat3(orientation)` where conventional GLM local-to-world conversion uses the matrix directly (`PhysicsBody.cpp:13, 26, 33, 51, 121, 140`). The helpers remain partially inverse to one another because both directions are swapped, but rendering uses `glm::toMat4(QuatRotation)` without that transpose. A rotated body's rendered orientation can therefore disagree with the space used for center of mass, contacts, and inverse inertia. Add golden tests for local/world point/vector conversion and inertia under a known 90-degree rotation before changing this high-risk convention.

Collision, broadphase, and numerical stability

The broadphase uses one-dimensional sweep-and-prune, rebuilding/sorting temporary endpoints every update. `Broadphase.cpp:184` allocates `2*N` endpoints with `_alloca`; large scenes can overflow the stack (also reported by MSVC C6255). Candidate generation is $O(N^2)$ in heavily overlapping scenes. The contact vector reserves `bodyCount2`, which can itself produce excessive memory demand. `CompareSAP` returns 1 rather than 0 for equal keys (`Broadphase.cpp:34`), violating the comparator contract expected by `qsort`.

Several math paths normalize vectors without checking length:

- coincident sphere centers in static/dynamic sphere contact (`PhysicsSystem.cpp:68, 128, 370`);
- GJK/contact direction (`PhysicsSystem.cpp:392` and related helpers);
- `GetOrtho` in `Math.h:139`;
- quaternion normalization in custom math (`Math.h:1032-1039`).

Coincident bodies are a normal collision input; a zero normal becomes NaN, contaminating impulses, positions, orientations, transforms, and ultimately GPU uniforms. Contact denominators and LCP pivots also lack epsilon guards. `ConstraintPenetration` computes a Baumgarte term proportional to C/dt (`ConstraintPenetration.cpp:117`) without protecting zero/very small delta.

The custom constraint `Mat/Vec` types store rows/elements in `std::vector`. Constructing small Jacobians and solver matrices therefore performs repeated heap allocation in a contact hot path and has poor locality. Replace them with fixed-size stack types or GLM matrices where dimensions are known.

No separate confirmed degrees/radians defect was found in the inspected active paths. Likewise, no general matrix multiplication defect was established beyond the specific, testable quaternion-space mismatch above; ambiguous transform conventions are classified as a highly likely bug rather than overstated as proven corruption.

Additional math defects

- `are_same_point` in `Math.h:1432` applies no `< epsilon` comparison to the Z difference; nonzero Z difference is treated as true. ParametricSurface, Prism, and Ring use the predicate for duplicate suppression.
- `BarryCentric` (`Math.cpp:99-102`) divides by a triangle determinant without a degenerate-triangle check.
- quaternion normalization divides by zero for a zero quaternion.
- solver divisions use exact/NaN checks rather than scale-aware epsilon and finite-value checks.

Performance Analysis

Rendering hot-path costs

Cost	Evidence	Priority
About 3 GiB shadow depth allocation	two $8192^2 \times 6 \times 32$ -bit resources	Immediate capacity/correctness concern
Four full scene submissions per normal frame	point shadow, cascade shadow, picking, final	High
Geometry shader layer amplification	six cubemap faces + five cascades	High; profile against per-face/multiview alternatives
Per-entity unconditional GL state calls	<code>UpdateRenderSetting</code>	Medium/High
Per-entity texture hash iteration/binds	<code>TexturesComponent::BindTextures</code>	Medium/High
Six UBO sub-updates	<code>SetupUBO</code>	Medium
Full picking render while idle	pass runs every frame; read only on click	High
Synchronous pick readback	<code>glReadPixels</code>	Medium, event-dependent stall
Duplicate point-light helper draws	nested group/all-lights loop	Medium
Texture cache misses/leaks	mismatched lookup/insertion key	High

Add debug counters for program/VAO/texture binds, draw calls, triangles, bytes uploaded, FBO reallocations, and readback time. Use an OpenGL debug context and KHR_debug labels/scopes so a RenderDoc capture maps back to passes and objects. Optimization should be driven by captures after startup correctness is fixed.

Physics hot-path costs

- worst-case $O(N^2)$ broadphase candidate scanning and $O(N^2)$ contact reserve;
- endpoint arrays rebuilt and sorted every frame rather than incrementally maintained;
- dynamic `std::vector` allocations throughout small constraint matrices;
- collision prediction mutates/integrates bodies twice, repeating quaternion/matrix work;
- transform matrices and inverse inertia transforms are recalculated repeatedly;
- manifolds/contacts use raw pointer collections with limited reuse.

A practical sequence is: fixed timestep and finite-value instrumentation first, then reusable frame arenas/fixed matrices, then incremental SAP or a spatial hierarchy after profiling realistic scene distributions.

CPU ray tracer

`SimpleRenderer.cpp:118` declares `pixelColor` outside a TBB parallel lambda, and the lambda writes it at line 154. Workers therefore race on one shared value, causing undefined behavior and wrong pixels. Make every pixel/sample accumulator lambda-local. `m_AccumulationData`, allocated at `SimpleRenderer.cpp:88`, is not deleted by the destructor (`SimpleRenderer.h:28`). Thread-local RNG engines avoid an engine data race, but only the initiating thread is explicitly seeded, so worker streams can start from identical default seeds.

Memory and Ownership Analysis

Confirmed lifetime problems

- Derived destructors in `Breakout` (`BreakoutApp.cpp:61–71`) and `GEngineEditor` (`SceneApp.cpp:39–48`) free asset/shader managers; `BaseApp::~BaseApp` repeats the operation. `AssetsManager` / `ShaderManager` delete values but do not clear maps (`AssetsManager.cpp:114–136`, `ShaderManager.cpp:36–44`).
- `Actor::operator=(Actor&&)` deletes `m_Parent` (`Actor.cpp:26–42`) although `Actor.h:224` marks it non-owning; moved children also retain their old parent pointer.
- `Texture` has no GPU-deleting destructor; the broken cache compounds both CPU wrapper and GPU leaks.
- physics shapes allocated in `_Scene::OnPhysics3DStart` are not owned/deleted.
- repeated physics start leaks the previous world; normal runtime stop does not invoke the existing physics stop.
- `SimpleRenderer` leaks its accumulation array.
- `Breakout` does not delete several directly allocated members (including ball/audio/editor camera), and `RayTracing`'s camera deletion is commented out.

Other C++20 safety observations

`SoundEvent::GetPlayState` and three `Bone` index helpers fall off non-void functions, confirmed by C4715. `_Entity::SetParent` dereferences the candidate parent before testing whether it is valid (`_Entity.cpp:109–111`). `_Entity::Transform() const` returns a `const Mat4&` bound to a temporary and asks for the legacy transform component (`_Entity.h:175`), while ECS entities use `Transform3DComponent`. The `_Scene::DestroyEntity(_Entity&)` overload at `_Scene.cpp:272–334` has its implementation commented out, so an lvalue can select a no-op overload while a by-value call destroys.

Move operations are not consistently `noexcept`, and `IndexBuffer` move assignment copies its CPU data instead of moving it. Manager singletons and raw IDs also obscure constness and exception guarantees. The code generally relies on assertions and does not provide transactional cleanup when initialization fails partway.

Threading Analysis

TBB is used in the CPU ray tracer and parallel container helpers. A dormant/raw-thread rendering branch also exists. The normal SDL event loop, scene update, physics update, OpenGL submission, ImGui rendering, and buffer readback execute on the main thread. No general task system with dependency tracking was found; `ThreadPool.h` exists but is not the primary frame scheduler.

Confirmed concurrency issue:

- shared `pixelColor` in the TBB pixel loop (`SimpleRenderer.cpp:118,154`) is a data race.

Architectural concerns:

- managers and ECS/render resources are not synchronized and must remain main-thread confined, but that contract is undocumented/unasserted;
- graphics wrappers do not record/assert context thread ownership;
- asynchronous asset loading is not implemented; adding it directly to current managers would race map access and could issue GL calls off-context;
- the dormant raw-thread branch has unclear lifecycle/first-entry state and should be removed or covered by explicit ownership and joining rules before enablement.

No mutex/condition-variable deadlock was found in the active application path. The present risk is unsynchronized shared state in TBB work and future misuse of implicitly main-thread-only systems, rather than excessive locking.

Bugs and Risks

Prioritized finding index

ID	Severity	Classification	Area	Short title
GE-001	Critical	Confirmed bug	ECS	OpenGL program ID used as fixed-vector index
GE-002	Critical	Confirmed bug	Ownership	manager resources double-freed at shutdown
GE-003	High	Confirmed bug	Packaging	clean builds cannot start: fonts not staged
GE-004	High	Confirmed bug	GPU upload	vertex sub-data byte count is wrong
GE-005	High	Confirmed bug	Framebuffer	invalid multisample resolve and incomplete targets
GE-006	High	Confirmed bug	Physics	zero-vector normalization propagates NaNs
GE-007	High	Highly likely bug	Physics/math	physics quaternion space disagrees with rendering
GE-008	High	Architectural risk	Physics/timing	clipped variable timestep and invalid time unit
GE-009	High	Confirmed bug	Input/loop	uninitialized mouse state; minimized event starvation
GE-010	High	Confirmed bug	Rendering	SpotLight branch accesses PointLight component
GE-011	High	Confirmed bug	Assets/GPU	texture cache mismatch and GPU texture leaks
GE-012	High	Confirmed bug	Ownership	Actor move deletes non-owning parent
GE-013	High	Confirmed bug	Shutdown	ImGui OpenGL shutdown after context destruction
GE-014	High	Confirmed bug	Concurrency	TBB ray-tracing pixel race
GE-015	High	Performance optimization opportunity	Rendering	3 GiB shadow maps and repeated full-scene passes
GE-016	High	Confirmed bug	ECS entity API	null-parent dereference/dangling transform reference
GE-017	Medium	Confirmed bug	C++ control flow	four non-void paths have no return

ID	Severity	Classification	Area	Short title
GE-018	Medium	Confirmed bug	Build/ODR	ShapeConvex implementation compiled twice
GE-019	Medium	Architectural risk	Physics performance	stack allocation and quadratic broadphase/contact capacity
GE-020	Medium	Confirmed bug	Physics ownership	worlds/shapes leak across stop/restart
GE-021	Medium	Confirmed bug	Rendering	PBR normal map decoded incorrectly
GE-022	Medium	Performance optimization opportunity	Picking	full pass every frame plus synchronous readback
GE-023	Medium	Confirmed bug	Math/geometry	duplicate-point Z predicate is malformed
GE-024	Medium	Confirmed bug	Entity lifecycle	lvalue DestroyEntity overload is a no-op
GE-025	Medium	Confirmed bug	Input	non-left mouse buttons never test true
GE-026	Medium	Architectural risk	Build/platform	advertised non-Windows targets are not buildable
GE-027	Medium	Architectural risk	Quality	no automated tests or coverage gate
GE-028	Low	Performance optimization opportunity	Rendering	redundant state, bind, lookup, and UBO calls
GE-029	Low	Confirmed bug	GL ownership	redundant index-buffer deletion
GE-030	Low	Confirmed bug	Ray tracer ownership	accumulation buffer leak

Detailed important findings

GE-001 — OpenGL program ID used as fixed-vector index

- **Severity / category:** Critical / Confirmed bug
- **File / function:** `GEngine/src/Scene/_Scene.cpp`, constructor lines 74–75; `Copy` line 113; `DuplicateEntity` line 215; `DestroyEntity` line 365; `render/light` group insertion lines 412 and 422. `GEngine/include/GEngine/Scene/_Scene.h:89–92`.
- **Problem and root cause:** `render/light` entity vectors are resized to five elements, then indexed as `renderId - 1`. `renderId` is an OpenGL program object name, not a dense engine-owned index and is not bounded by five.
- **Runtime consequence:** out-of-bounds read/write, heap corruption, crashes, or incorrect render grouping.
- **Reproduction:** create enough GL programs that a material program name exceeds 5, then add/copy/destroy an entity using that program.
- **Recommended fix:** key groups by a stable engine `ShaderHandle` in an `unordered_map`, or maintain a validated dense indirection table. Never derive storage indices from API object names. Add add/copy/destroy tests with sparse IDs.
- **Fix risk:** Medium; affects serialization/group iteration and both render/light paths.

GE-002 — Global assets/shaders are double-freed

- **Severity / category:** Critical / Confirmed bug
- **File / function:** `Breakout/src/BreakoutApp.cpp:61–71`, `GEngineEditor/src/SceneApp.cpp:39–48`, `GEngine/src/Core/BaseApp.cpp:26–31`, `AssetsManager.cpp:114–136`, `ShaderManager.cpp:36–44`.
- **Problem and root cause:** derived and base destructors both delete manager-owned raw pointers, and free methods do not erase/clear the maps.
- **Runtime consequence:** use-after-free/double deletion during orderly application exit. GPU wrapper behavior can make this driver-dependent.
- **Reproduction:** repair the font startup path, launch Breakout or GEngineEditor, then close normally under AddressSanitizer/Application Verifier.
- **Recommended fix:** make one layer the sole owner; use manager destructors/`unique_ptr`, make shutdown idempotent, clear containers after destruction, and remove derived free calls.
- **Fix risk:** Medium; shutdown ownership changes can expose hidden external references.

GE-003 — Clean builds cannot start

- **Severity / category:** High / Confirmed bug
- **File / function:** `GEngine/src/Windows/ImGuiWindow.cpp:10-11`; Premake post-build blocks and `tools/run.bat`.
- **Problem and root cause:** fonts are loaded from a working-directory-relative path not created by the build. Missing post-build source directories fail silently.
- **Runtime consequence:** all four applications assert before their normal content loop in Debug and Release.
- **Reproduction:** clean export → generate/build → run from `bin/<cfg>/<app>`.
- **Recommended fix:** central executable-relative asset locator; explicit build asset-copy target; fail packaging on missing assets; add a clean-package launch smoke test.
- **Fix risk:** Medium because all existing relative asset paths should migrate together.

GE-004 — VertexBuffer uploads one quarter of float data

- **Severity / category:** High / Confirmed bug
- **File / function:** `GEngine/src/Mesh/VertexBuffer.cpp:45`, `VertexBuffer::SetData`.
- **Problem and root cause:** OpenGL expects a byte count, but `vector<float>::size()` is passed directly.
- **Runtime consequence:** most dynamic vertex data remains stale/uninitialized; geometry can corrupt or disappear.
- **Reproduction:** allocate a buffer, update multiple float vertices with `SetData`, inspect in RenderDoc or read back the buffer.
- **Recommended fix:** pass `data.size() * sizeof(data[0])`; use a byte/span API to make units explicit; add a GL integration test.
- **Fix risk:** Low.

GE-005 — Framebuffer creation/resolve violates OpenGL rules

- **Severity / category:** High / Confirmed bug
- **File / function:** current `GEngine/src/Core/RenderTarget.cpp:165-272`, `408-414`, `697-792`, `794-907`.
- **Problem and root cause:** multisample blit uses `GL_LINEAR`; draw attachment 1 is enabled without attachment creation; rectangular targets allocate width for both dimensions; integer picking uses linear filtering.
- **Runtime consequence:** `GL_INVALID_OPERATION`, incomplete FBOs, failed/undefined resolve, distorted or out-of-range picking.
- **Reproduction:** proceed past font init with an OpenGL debug context; validate FBO status and errors on a non-square viewport/MSAA resolve.
- **Recommended fix:** `GL_NEAREST` resolve, attach only created targets, use height, use nearest integer filtering, centralize descriptor validation and assert completeness for every FBO.
- **Fix risk:** Medium; current files are in-progress owner changes, so integrate with that work carefully.

GE-006 — Coincident/degenerate collision inputs generate NaNs

- **Severity / category:** High / Confirmed bug
- **File / function:** `GEngine/include/GEngine/Physics/PhysicsSystem.cpp:68,128,370,392`; `Math.h:139,1032-1039`; constraint divisions.
- **Problem and root cause:** zero vectors/quaternions and near-zero denominators are normalized/divided without epsilon/finite guards.
- **Runtime consequence:** NaN velocities/transforms spread through solver and render uniforms; bodies disappear or simulation explodes.
- **Reproduction:** spawn two sphere centers at identical coordinates with zero relative separation, then step physics.
- **Recommended fix:** deterministic fallback normals, `length2 > epsilon2` checks, finite assertions, minimum pivot/denominator policy, and degenerate-case tests.
- **Fix risk:** Medium; fallback direction affects stacking behavior.

GE-007 — Physics and rendering use opposite quaternion matrix convention

- **Severity / category:** High / Highly likely bug

- **File / function:** `GEngine/include/GEngine/Physics/PhysicsBody.cpp:13,26,33,51,121,140`; ECS render transform composition.
- **Problem and root cause:** physics transposes GLM's quaternion matrix for local-to-world and effectively reverses world-to-local, while rendering uses the normal GLM quaternion matrix.
- **Runtime consequence:** rotated collision geometry, center of mass, inertia, and contact arms can disagree with visible orientation.
- **Reproduction:** rotate an asymmetric box 90° and compare rendered axes with support points/contact response along each world axis.
- **Recommended fix:** define/document one column-vector convention; add golden conversion tests; then update all body-space/world-space and inertia conversions atomically.
- **Fix risk:** High because changing convention can invert already-compensating code.

GE-008 — Frame time and physics step are invalid/non-deterministic

- **Severity / category:** High / Architectural risk
- **File / function:** `GEngine/src/Core/BaseApp.cpp:293-315`; `RigidBodySimulation.cpp:663-668`; `ConstraintPenetration.cpp:117`.
- **Problem and root cause:** microseconds are multiplied by 1,000 for a value called milliseconds; sleep duration replaces measured duration; long frames are dropped; no fixed accumulator; solver divides by dt.
- **Runtime consequence:** input scaling errors, frame-rate-dependent physics, lost simulation time, tunneling/solver instability, nondeterministic results.
- **Reproduction:** compare body trajectories at 30/60/144 Hz and with injected 50 ms frame stalls.
- **Recommended fix:** strongly typed seconds; fixed-step accumulator (for example 1/120 s); backlog cap with retained remainder; render interpolation; explicit zero-dt behavior.
- **Fix risk:** High because gameplay tuning will change.

GE-009 — Input UB and minimized-loop starvation

- **Severity / category:** High / Confirmed bug
- **File / function:** `GEngine/include/GEngine/Managers/InputManager.h:95-102`; `InputManager.cpp:194`; `BaseApp.cpp:285`.
- **Problem and root cause:** `m_IsRelative` is not initialized before use. Separately, minimized state skips event pumping and sleep.
- **Runtime consequence:** undefined mouse behavior; minimized app busy-spins and cannot process restoration or quit.
- **Reproduction:** launch under UB instrumentation; minimize the window and observe one CPU core plus no restore processing.
- **Recommended fix:** in-class initialize all input state; always poll events, throttle when minimized, and render/update conditionally after polling.
- **Fix risk:** Low.

GE-010 — SpotLight rendering fetches PointLightComponent

- **Severity / category:** High / Confirmed bug
- **File / function:** current `GEngine/include/GEngine/Core/RenderSystem.cpp:224-227`; legacy spot-light path around lines 348-351.
- **Problem and root cause:** the SpotLight branch requests `PointLightComponent` from the entity.
- **Runtime consequence:** EnTT assertion/invalid component access or incorrect lighting when a spot light is rendered.
- **Reproduction:** create an entity with SpotLightComponent but no PointLightComponent and submit the affected pass.
- **Recommended fix:** fetch SpotLightComponent and centralize typed light extraction shared by legacy/ECS paths.
- **Fix risk:** Low.

GE-011 — Texture cache never hits and GPU objects leak

- **Severity / category:** High / Confirmed bug

- **File / function:** GEngine/src/Managers/AssetsManager.cpp:25,35,114–136; GEngine/include/GEngine/Assets/Textures/Texture.h:61–102.
- **Problem and root cause:** lookup and insertion use different keys; failed insertions leave returned raw allocations untracked; Texture lacks glDeleteTextures destruction.
- **Runtime consequence:** repeat disk decode, CPU/GPU memory growth, VRAM exhaustion, and slow asset use.
- **Reproduction:** load the same texture path repeatedly and watch loader calls/GL texture count.
- **Recommended fix:** canonical std::filesystem::path key, unique_ptr/shared asset handle, insertion-result handling, and explicit owning vs non-owning texture types.
- **Fix risk:** Medium due to framebuffer attachment wrappers and shared users.

GE-012 — Actor move assignment deletes a non-owning parent

- **Severity / category:** High / Confirmed bug
- **File / function:** GEngine/src/Core/Actor.cpp:26–42; ownership declaration Actor.h:224.
- **Problem and root cause:** move assignment calls delete m_Parent although parent is non-owning, then moves children without repairing their parent links.
- **Runtime consequence:** parent use-after-free/double delete and corrupted hierarchy after move assignment.
- **Reproduction:** move-assign a child actor that already has a parent, then traverse/destroy the hierarchy.
- **Recommended fix:** never delete observer pointers; reset/rebind hierarchy explicitly or make Actor non-movable while attached; use stable entity handles.
- **Fix risk:** Medium.

GE-013 — Graphics context is destroyed before ImGui backend shutdown

- **Severity / category:** High / Confirmed bug
- **File / function:** GEngine/src/Windows/SDLWindow.cpp:106–115; ImGuiWindow destructor.
- **Problem and root cause:** window/context destruction precedes deletion of the object that calls ImGui OpenGL backend shutdown.
- **Runtime consequence:** GL cleanup without a valid context, driver errors/crash/leaks, especially with platform viewports.
- **Reproduction:** normal close with GL debug output after startup is repaired.
- **Recommended fix:** make context current; destroy app GPU resources and ImGui backends/context; then delete GL context and SDL window; close controllers and call TTF/SDL quit once.
- **Fix risk:** Medium.

GE-014 — TBB ray tracer has a shared pixel accumulator

- **Severity / category:** High / Confirmed bug
- **File / function:** GEngine/src/Core/SimpleRenderer.cpp:118,154.
- **Problem and root cause:** pixelColor is declared outside the parallel lambda and written by multiple workers.
- **Runtime consequence:** C++ data race (undefined behavior), nondeterministic colors, flicker/corruption.
- **Reproduction:** run RayTracing with TBB under ThreadSanitizer on a supported toolchain or compare repeated checksums.
- **Recommended fix:** declare accumulator inside the per-pixel lambda; keep only disjoint pixel writes; seed each worker stream intentionally.
- **Fix risk:** Low.

GE-015 — Shadow configuration and multipass submission exceed practical budgets

- **Severity / category:** High / Performance optimization opportunity
- **File / function:** current BaseApp.cpp:121–124; RigidBodySimulation frame at lines 726–850; layered shadow shaders.
- **Problem and root cause:** two approximately 1.5 GiB depth resources and four full-scene submissions per frame, with 6/5-layer geometry amplification.
- **Runtime consequence:** allocation failure on common GPUs, paging/stutter, high fill/geometry cost, slow resize and startup.
- **Reproduction:** inspect GL allocation/error/debug output on 4–8 GiB GPUs; capture frame after font fix.

- **Recommended fix:** start with 1024–2048 face/cascade budgets, configurable quality tiers, per-pass culling, cached static shadows, on-demand picking, and measured cascade count/resolution.
- **Fix risk:** Medium; visual quality must be evaluated.

GE-016 — ECS parent/transform APIs return invalid results

- **Severity / category:** High / Confirmed bug
- **File / function:** `GEngine/src/Scene/_Entity.cpp:109–111`; `GEngine/include/GEngine/Scene/_Entity.h:175`.
- **Problem and root cause:** `SetParent` calls `GetUUID()` before validating parent. `Transform()` `const` returns a reference to a temporary and requests the wrong transform component type.
- **Runtime consequence:** null-scene dereference, dangling reference, EnTT assertion/UB.
- **Reproduction:** call `SetParent({})`; call `const Transform()` on a normal ECS entity.
- **Recommended fix:** validate first; return transform by value or a reference to stored component data; use `Transform3DComponent`; add parent-cycle/validity tests.
- **Fix risk:** Low/Medium due to API signature changes.

GE-017 through GE-030 — concise evidence and treatment

ID	Evidence / consequence	Recommended fix	Fix risk
GE-017	<code>SoundEvent.cpp:95</code> and <code>Bone.h:95,106,117</code> fall off non-void functions (MSVC C4715), producing undefined return values.	Return a defined state/index or assert+sentinel; add boundary tests.	Low
GE-018	<code>Shapes/Diamond.h:4</code> includes <code>Physics/ShapeConvex.cpp</code> while Premake compiles that <code>.cpp</code> ; linker emits LNK4006 and ignores duplicates.	Include the header, compile implementation exactly once.	Low
GE-019	<code>Broadphase.cpp:184</code> unbounded <code>_alloca</code> ; quadratic overlap scan/contact reserve; comparator returns nonzero equality.	reusable heap/frame arena, valid strict comparator, capacity limit, incremental/spatial broadphase.	Medium
GE-020	<code>_Scene.cpp:438–527</code> raw world/shapes; runtime stop omits physics stop; bodies do not own shapes.	<code>unique_ptr<PhysicsWorld></code> and shared/unique shape assets with explicit teardown/restart.	Medium
GE-021	<code>pbr_cascade_shadow.frag:43–44</code> does not remap normals and uses wrong UV source.	remap, normalize, and use passed/tiled UV; add flat-normal/material render test.	Low
GE-022	mouse-pick scene rendered every frame and click readback uses synchronous <code>glReadPixels</code> .	invalidate/on-demand pass; scissor/PBO/fence if continuous.	Medium
GE-023	<code>Math.h:1432</code> tests raw absolute Z difference as boolean, breaking equality.	compare all three axes to epsilon or squared distance.	Low
GE-024	<code>_Scene.cpp:272–334</code> lvalue-reference destroy overload is commented/no-op.	remove overload ambiguity or implement one canonical handle-based destroy.	Medium
GE-025	<code>MouseState::GetButtonValue</code> compares <code>(mask & state) == 1</code> , so only bit 0 can pass.	test <code>!= 0</code> ; cover every SDL mouse button.	Low
GE-026	unguarded CRT call, Windows binaries/paths, and no validated non-Windows dependencies contradict Premake filters.	explicitly declare Windows-only or build CI-supported platform dependency paths.	Medium
GE-027	no test framework, <code>enable_testing</code> , <code>add_test</code> , or executable test target.	layered unit/headless GL/smoke tests and CI gates.	Medium
GE-028	unconditional per-entity GL state, texture binds, six UBO uploads, repeated cached shader hash lookups.	render-state cache, material sorting, packed per-frame UBO, pass descriptors.	Medium
GE-029	Geometry and IndexBuffer both delete the EBO ID. OpenGL tolerates deleting a nonexistent name, but ownership is ambiguous.	one move-only owner; Geometry stores wrappers, not duplicate raw IDs.	Low
GE-030	<code>SimpleRenderer.cpp:88</code> allocates accumulation data; destructor in <code>SimpleRenderer.h:28</code> only frees image data.	<code>std::vector<Vec4f></code> or <code>unique_ptr<Vec4f[]></code> .	Low

The index above supplies severity and classification for these secondary findings. Their root causes and direct reproduction methods are:

ID	Root cause	Reproduction method
GE-017	Sentinel/error paths reach the closing brace without a defined return.	Request an absent FMOD playback state or an animation time beyond the final key and inspect the returned value under compiler diagnostics.
GE-018	A header includes an implementation <code>.cpp</code> that the project also compiles.	Rebuild GEngine and observe LNK4006 for ShapeConvex methods.
GE-019	Per-frame temporary capacity is proportional to N^2 , and endpoints use unbounded stack allocation.	Create a large, mutually overlapping body set and monitor stack/contact memory and candidate count.
GE-020	Raw ownership is split among Scene, World, Body, and Shape with no stop/restart contract.	Start/stop/start ECS physics and inspect live allocations; then destroy the scene.
GE-021	Tangent normals are treated as already signed and the function ignores its UV parameter.	Render a flat normal map and a material with texture tiling; compare normals/channels in a GPU capture.
GE-022	Picking is scheduled unconditionally, while selection is event-driven and readback is immediate.	Capture several idle frames, then click and measure the <code>glReadPixels</code> call.
GE-023	The Z term lacks the same epsilon comparison used by X/Y.	Compare points equal in X/Y but separated in Z and inspect duplicate suppression in Ring/Prism/ParametricSurface.
GE-024	An exact lvalue-reference overload exists with its body commented out.	Pass a named <code>_Entity</code> variable to <code>DestroyEntity</code> and verify the registry still contains it.
GE-025	A bit mask is compared to literal 1 instead of nonzero.	Press SDL mouse buttons whose mask is 2, 4, or higher and query the value.
GE-026	Build filters were added without portable entry code, dependencies, or CI validation.	Generate a non-Windows project and compile/link from a clean checkout.
GE-027	No automated test target/framework or CI coverage contract exists.	Search project definitions and run CTest; no tests are registered.
GE-028	Render state is issued from entity code without a last-bound state cache or material sort.	Capture a frame and count identical adjacent state/bind calls and the six UBO sub-updates.
GE-029	Both aggregate and member wrapper consider the same EBO name owned.	Destroy a Geometry with an IndexBuffer under a GL debug/object-lifetime trace.
GE-030	The manual destructor omits the second manually allocated array.	Repeatedly construct/destroy SimpleRenderer under a heap/leak profiler.

Static Analysis

No `.clang-tidy` configuration exists, and `clang-tidy/clang` were not installed, so no Clang analysis was attempted or installed. `cppcheck` was also unavailable. MSVC native code analysis was run on a clean Debug engine rebuild with C++ Core Check enabled.

Project warning counts from that run (third-party warnings excluded when interpreting):

Code	Count	Meaning / notable evidence
C4244	48	numeric conversion/narrowing
C26495	46	member variables not initialized
C4267	23	size type narrowing
C4715	4	non-void path has no return: SoundEvent and Bone helpers
C26820	2	expensive copies in <code>Character.cpp:40</code> , <code>CameraRig.cpp:64</code>

Code	Count	Meaning / notable evidence
C26439	2 project-relevant	move/noexcept guidance, including <code>IndexBuffer.cpp:14</code>
C6255	1	<code>_alloca</code> stack-overflow risk at <code>Broadphase.cpp:184</code>
C26800	1 project-relevant	moved-from object warning at <code>Events/Event.h:145</code>

The full Release build's 956 warnings include engine, application, generated/vendored, narrowing, duplicate-definition, and post-build diagnostics. Establish `/W4`, separate third-party warning suppression, then ratchet the project-owned warning baseline to zero. Add AddressSanitizer for supported MSVC/Clang configurations and a GL debug callback. ThreadSanitizer requires a compatible non-MSVC target/toolchain and would be especially valuable for RayTracing.

Tests and Coverage Gaps

No executable unit/integration test project, CTest registration, Catch2, GoogleTest, or other active test macro was found. `GEngineEditor/src/test.cpp` is commented experimental code, and `SceneTest` is an application scene, not an automated test. Consequently, there is no coverage report or regression gate.

Highest-value initial test set:

1. **Pure math:** quaternion/matrix convention, zero normalization, barycentric degeneracy, duplicate-point predicate, transform order.
2. **Physics deterministic fixtures:** falling body at multiple render rates; coincident spheres; static/dynamic and static/static; rotated box support/inertia; zero/large dt; finite-state assertion after every step.
3. **ECS lifecycle:** sparse shader handles, create/copy/duplicate/destroy, lvalue/rvalue overloads, parent null/cycles, runtime start/stop/restart.
4. **Ownership:** manager free twice is harmless, asset cache returns one handle, move semantics preserve hierarchy, physics shapes/worlds are reclaimed.
5. **Headless/OpenGL integration:** buffer upload byte equality, each FBO completeness, non-square picking, MSAA resolve, resize zero/nonzero, GL error-free destruction.
6. **Clean package smoke:** generate/build/stage and launch each executable long enough to enter its first frame; verify required DLL/assets before launch.
7. **Shader/render golden tests:** flat normal texture, cascade selection, entity ID output, light component type, and a small reference image with tolerant comparison.
8. **Concurrency:** repeated deterministic ray-tracing checksum with worker counts 1 and N; sanitizer job where supported.

Architecture Improvements

Target ownership architecture

```
Application (unique owner)
+-- EngineContext
|   +-- WindowSystem -> Window -> GLContext -> ImGuiLayer
|   +-- AssetRegistry<stable handles>
|   +-- RenderDevice<move-only GL resources>
|   +-- Input/Event services
+-- Scene (ECS)
|   +-- components store handles/values, not owning raw pointers
|   +-- PhysicsWorld owns bodies; ShapeRegistry owns immutable shapes
+-- Renderer
    +-- extracts immutable RenderWorld
    +-- schedules explicit passes/resources
    +-- sorts/batches submissions and caches state
```

Key design changes:

- replace global static managers with an explicit `EngineContext` passed to owners;
- represent assets/shaders/materials with generation-checked handles;
- use one scene model and one light extraction path;
- define an explicit `RenderFrame`/render graph with target descriptors, dependencies, and resize ownership;
- separate graphics device resources from asset metadata and context-independent decoded data;
- make physics queries pure and physics stepping fixed/deterministic;
- eliminate API overloads whose reference category changes semantics;
- centralize runtime path resolution and packaging manifest;
- encode units (`Seconds`, `pixels`, `bytes`) in types/APIs.

Instrumentation

Before performance rewrites, expose per-frame CPU update/physics/render timings, per-pass GPU timer queries, draw/triangle/bind/upload/readback counters, live GL resource bytes/counts, physics candidates/contacts/iterations, and allocation counts. Emit finite-value assertions for physics state and `KHR_debug` messages with object labels in `Debug`.

Prioritized Remediation Roadmap

Phase 1: Critical correctness, crashes, undefined behavior, memory/resource lifetime

1. Replace program-ID vector indexing (GE-001) and add sparse-ID lifecycle tests.
2. Make manager/resource shutdown single-owner and idempotent (GE-002); fix context/ImGui destruction order (GE-013).
3. Establish/stage a canonical asset root so clean applications enter the frame loop (GE-003).
4. Fix vertex byte count, all missing returns, mouse initialization/mask, parent/transform invalid APIs, and no-op destroy overload.
5. Repair framebuffer completeness/resolve/dimensions/filtering and enable OpenGL debug validation.
6. Guard collision normalization/divisions and instrument all physics state for finiteness.
7. Make the TBB pixel accumulator local and reclaim the accumulation buffer.

Exit gate: clean `Debug`/Release build, zero project-owned high-confidence UB warnings, all four applications complete first frame and clean shutdown under sanitizer/debug GL, core regression tests green.

Phase 2: Architecture and ownership

1. Introduce explicit EngineContext and deterministic shutdown; remove duplicate global frees.
2. Convert CPU ownership to `unique_ptr`/value types and relationships to stable handles.
3. Implement move-only RAI for every GL object and distinguish owning textures from attachment views.
4. Unify legacy/ECS render-world and light extraction or publish a deprecation migration plan.
5. Define asset manifest/root and configuration-specific imported dependency targets.

Phase 3: Rendering correctness and performance

1. Correct PBR normal decoding/UVs and SpotLight access with shader/render fixtures.
2. Reduce shadow defaults, add quality settings and per-pass culling.
3. Render picking on demand; pipeline readback if continuous picking is required.
4. Add render-pass descriptors, state cache, material sorting, packed per-frame buffers, and resource metrics.
5. Separate native-window and editor-viewport resize state; debounce allocations.
6. Capture representative GPU frames and optimize measured geometry/fill/state bottlenecks.

Phase 4: Physics correctness and performance

1. Adopt fixed-step accumulator and typed seconds; add cross-frame-rate trajectory tests.
2. Validate/fix quaternion-space convention atomically with golden tests.
3. Make collision prediction pure; define continuous collision/TOI progression.
4. Replace unbounded stack/quadratic pre-reserve behavior and invalid comparator.
5. Replace tiny heap-backed solver matrices, cache repeated transforms/inertia, and profile broadphase choices.
6. Make world/shape ownership explicit and runtime start/stop/restart safe.

Phase 5: Testing, maintainability, and cleanup

1. Add unit, GL integration, packaging smoke, image, sanitizer, and deterministic concurrency tests.
2. Enable `/W4`, isolate vendor warnings, eliminate warning baseline, and add CI code analysis.
3. Choose explicit Windows-only support or validate other platforms in CI.
4. Remove committed build products where practical and document dependency provenance/versions/licenses.
5. Replace hard-coded build helpers; optionally add a first-class CMake build, but avoid maintaining two untested authorities.

Build / Run / Debug Cheat Sheet

```
# Generate
& .\vendor\bin\premake\premake5.exe vs2022

# Build
msbuild .\GEngine.sln /m /nologo /p:Configuration=Debug /p:Platform=x64
msbuild .\GEngine.sln /m /nologo /p:Configuration=Release /p:Platform=x64

# Run (intended working directory; currently blocked by GE-003 in a clean checkout)
Push-Location .\bin\Debug\RigidBodySimulation
.\RigidBodySimulation.exe
Pop-Location

# Run another product
Push-Location .\bin\Debug\GEngineEditor
.\GEngineEditor.exe
Pop-Location

# MSVC native analysis
msbuild .\GEngine.sln /t:GEngine:Rebuild /m /nologo `
  /p:Configuration=Debug /p:Platform=x64 `
  /p:RunCodeAnalysis=true /p:EnableCppCoreCheck=true

# Repository state
git status --short
```

Debugging priorities after asset staging is fixed:

1. enable an OpenGL debug context/callback and break on high-severity messages;
2. run normal shutdown under AddressSanitizer/Application Verifier;
3. test coincident physics bodies with finite assertions;
4. use RenderDoc to verify FBO completeness, actual draw counts, shadow storage, and buffer contents;
5. compare ray-tracing checksums at different TBB worker counts.

Verification Record

- Markdown report path verified during audit handoff.
- Build commands above were executed against a clean export of the audited revision for Debug and Release.
- All four applications were launched in both configurations from their intended target directories; all produced the same font assertion before first frame.
- Important confirmed claims were traced to the cited code and, where feasible, supported by compiler/linker/runtime output.
- No production source was changed. The only intended audit artifacts are `docs/audit/CODEBASE_AUDIT.md` and `docs/audit/CODEBASE_AUDIT.pdf`.